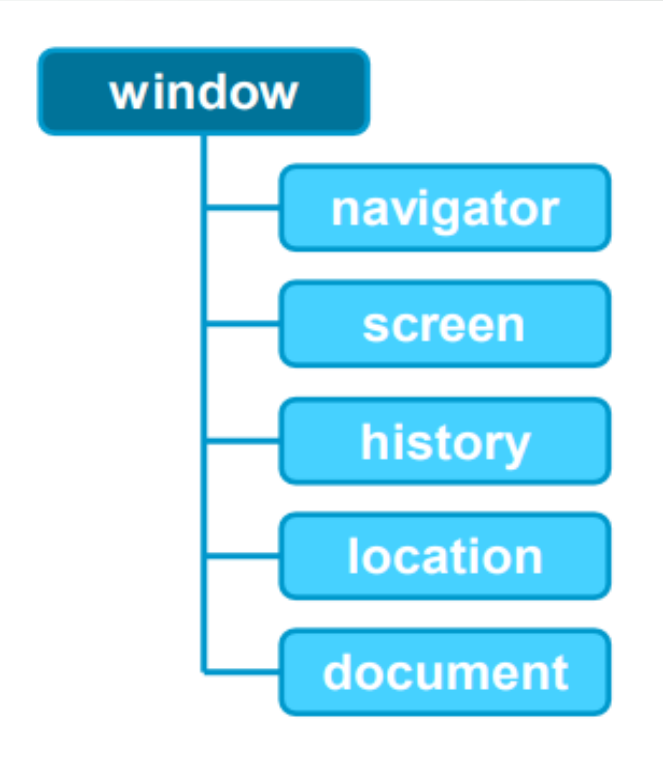
Browser Object Model (BOM)

ITI - Assiut Branch

Eng. Sara Ali

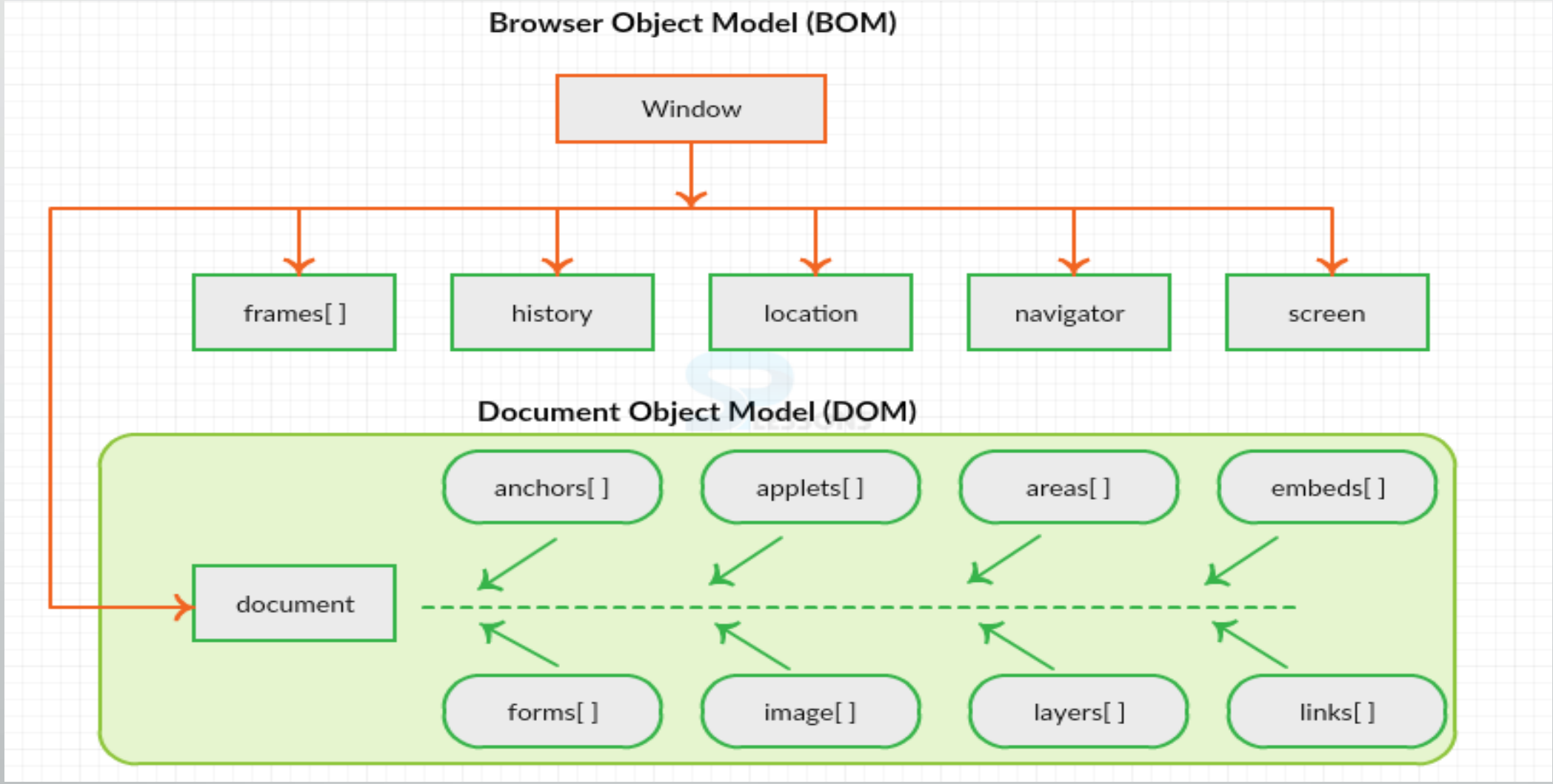
What is BOM?

- BOM Stands for Browser Object Model.
- BOM covers objects which relate to the browser.
- The BOM's root is the window object, which displays properties of the current open window or tab. It has several children (navigator - screen - history - location -document)



- Using the BOM, developers can move the window, change text in the status bar, and perform other actions that do not directly relate to the page content.
- The DOM, which relates to all things pertaining to the document, resides within the BOM.

BOM Model


Browser Object Model (BOM)
Window
frames[ ]
history
location
navigator
screen
Document Object Model (DOM)
document
anchors[ ]
applets[ ]
areas[ ]
embeds[ ]
forms[ ]
image[ ]
layers[ ]
links[ ]

BOM Model (Cont.)

- Here is a brief description of each element on the tree.
  - **Window** : the top-level object; has properties that apply to the entire window.
  - **Document** : Contains all HTML elements of the current page.
  - **Frames** : Any frames embedded within the current page.
  - **History** : Contains a stack of sites visited in the current session.
  - **Location** : Stores URL of the current page.
  - **Navigator** : Stores information about the browser being used, and the OS used by the browser.
  - **Screen** : Display properties of the user's computer or device.

Window Object

- The window object represents the current open window on a browser.
- Object Model Reference:
  - window
- To reference its properties & methods:

[window.]property
[window.]method

Window Object (Cont.)

- **Window Properties**

Property	Description	Syntax
name	Return or set a window's name	window.name
closed	Returns whether a window has been closed	window.closed
innerHeight	Returns the inner Height of a window's content area	window.innerHeight
innerWidth	Returns the inner width of a window's content area	window.innerWidth
outerHeight	Returns the outer height of a window, including toolbars/scrollbars	window.outerHeight
outerWidth	Returns the outer width of a window, including toolbars/scrollbars	window.outerWidth

Window Object (Cont.)

- **Window Properties (Cont.)**

Property	Description	Syntax
screenLeft = screenX	Returns the horizontal coordinate of the window relative to the screen	window.screenLeft
screenTop = screenY	Returns the vertical coordinate of the window relative to the screen	window.screenY
pageXOffset = scrollX	Returns the pixels the current document has been scrolled (horizontally) from the upper left corner of the window	window.pageXOffset
pageYOffset = scrollY	Returns the pixels the current document has been scrolled (vertically) from the upper left corner of the window	window.pageYOffset

Window Object (Cont.)

- **Window Properties (Cont.)**

Property	Description	Syntax
document	Reference to the current document object.	window.document
screen	Reference to the screen object of JavaScript	window.screen
history	Reference to the History object of JavaScript	window.history
navigator	Reference to the browser application	window.navigator
location	Reference to the Location object of JavaScript	window.location

Window Object (Cont.)

- **Window Methods**

Method	Description	Syntax
alert()	Displays an alert box with a message and an OK button	window.alert("Hello")
confirm()	Displays a dialog box with a message and an OK, returning true, and a Cancel, returning false	window.confirm("Do you want to exit")
prompt()	Displays a dialog box that prompts the user for input	name=prompt("Please enter your name", "")
open()	Opens a new browser window	window.open(url, name, properties)
close()	Close a specified window	window.close()

Window Object (Cont.)

- **Window Methods (Cont.)**

Method	Description	Syntax
focus()	Sets focus to the current window	window.focus();
blur()	Removes focus from the current window	window.blur();
getSelection()	Returns a Selection object representing the range of text selected by the user	window.getSelection();
print()	Print the contents of the specified window.	window.print();

Window Object (Cont.)

- **Window Methods (Cont.)**

Method	Description	Syntax
moveTo(h,v)	Moves the window to horizontal and vertical position relative top-left of screen.	window. moveTo(,)
moveBy(h,v)	Moves the window by + or - horizontal and vertical pixels:	window.moveBy(,)
resizeTo(h,v)	Changes the size of the window to horizontal and vertical number of pixels.	window.resizeTo(,)
resizeBy(h,v)	Changes the size of the window by + or - horizontal and vertical pixels.	window.resizeBy(,)
scrollTo(h,v)	Scrolls the document in the current window or frame to horizontal and vertical pixel postions from top of document	window.scrollTo(,)
scrollBy(h,v)	Scrolls the document in the current window or frame by + or - horizontal and vertical pixel from current position.	window.scrollBy(,)

Window Object (Cont.)

- **Window Methods (Cont.)**

Method	Description	Syntax
setInterval(expression, interval)	Evaluates an expression at specified intervals	var t = window.setInterval(*"alert()",500*) Or window.setInterval(*funcName,500*)
clearInterval(interval_Obj_Name)	Used to clear a time interval set using the above method	window.clearInterval(t)
setTimeout()	Used to execute an expression or function after a time interval (in millisecond).	var t = window.setTimeOut(*exp,time_interval*)
clearTimeout()	Used to clear a timeout set using the above method	window. clearTimeout(t)

Screen

- The window.screen object contains information about the user's screen such as resolution (i.e. width and height of the screen), color depth, pixel depth, etc.
- Since window object is at the top of the scope chain, so properties of the window.screen object can be accessed without specifying the window. prefix, for example window.screen.width can be written as screen.width

Screen (Cont.)

- **Screen Properties**

Property	Description
availHeight	Returns the height of the screen (excluding the Windows Taskbar)
availWidth	Returns the width of the screen (excluding the Windows Taskbar)
height	Returns the total height of the screen
width	Returns the total width of the screen
pixelDepth	Returns the color resolution (in bits per pixel) of the screen
colorDepth	Returns the bit depth of the color palette for displaying images. Color depth is the number of bits used to represent the color of a single pixel. color depth of 8 can produce 256 colors (2^8)

Navigator

- The navigator object contains information about the browser.
- The navigator property of a window (i.e. window.navigator) is a reference to a Navigator object.
- **Navigator Properties**

Property	Description
appName	Returns the name of the browser
appVersion	Returns the version information of the browser
cookieEnabled	Determines whether cookies are enabled in the browser
geolocation	Returns a Geolocation object that can be used to locate the user's position
language	Returns the language of the browser
onLine	Determines whether the browser is online
platform	Returns for which platform the browser is compiled

Location

- The location object contains information about the current URL.
- The location object is part of the window object and is accessed through the window.location property.

Location Properties

- Assume that the current URL is http://www.example.com:4097/test.htm#part2

Property	Description	Example
hash	Sets or returns the anchor part (#) of a URL	location.hash; //#part2 location.hash = ”part4”
host	Sets or returns the hostname and port number of a URL	location.host; //www.example.com:4097
hostname	Sets or returns the hostname of a URL	Location.hostname; //www.example.com
href	Sets or returns the entire URL	Location.href; //http://www.example.com/test.htm#part2

Location (Cont.)

- **Location Properties (Cont.)**
  - Assume that the current URL is http://www.example.com:4097/test.html#part2

Property	Description	Example
origin	Returns the protocol, hostname and port number of a URL	Location.origin; //http://www.example.com:4097
pathname	Sets or returns the path name of a URL	Location.pathname; //test.html
port	Sets or returns the port number of a URL	Location.port; //4097
protocol	Sets or returns the protocol of a URL	Location.protocol; //http
search	Sets or returns the querystring part of a URL	Assume that the current URL is https://www.w3schools.com/submit.htm?email=someone@example.com. Location.search; //?email=someone@example.com

Location (Cont.)

- **Location Methods**

Name	Description
replace(URL)	Replaces the current document with a new one
assign(URL)	Loads a new document. almost the same as replace method. The difference is that it creates an entry in the browser's history list, while replace() doesn't
reload()	Reloads the current document

- Use the assign() method if you want to load a new document, and the option to navigate back to the original document.

History

- The history object contains the URLs visited by the user (within a browser window).
- The history object is part of the window object and is accessed through the window.history property.
- **History Properties**

Property	Description
length	Returns the number of URLs in the history list

History (Cont.)

- **History Methods**

Method	Description
back()	Loads the previous URL in the history list
forward()	Loads the next URL in the history list
go()	Loads a specific URL from the history list

Event Object

- An event is something that happens when user interact with the web page, such as when he clicked a link or button, entered text into an input box, pressed key on the keyboard, submits a form, etc.
- When an event occur, you can use a JavaScript event handler to detect them and perform specific task or set of tasks. By convention, the names for event handlers always begin with the word "on", so an event handler for the click event is called onclick, similarly an event handler for the load event is called onload, event handler for the blur event is called onblur, and so on.
- HTML DOM events allow JavaScript to register different event handlers on elements in an HTML document.
- When an event occurs an object of Event is created to hold some additional information about the occurred event.

Event Object (Cont.)

- **Event object properties**

Property	Description
Target	The element that fired the event (IE old versions = srcElement)
type	Type of event
bubbles	Returns whether or not a specific event is a bubbling event
cancelable	Returns whether or not an event can have its default action prevented

MouseEvent object

- **MouseEvent object properties**

Property	Description
screenX	Returns the horizontal coordinate of the mouse pointer, relative to the screen, when the mouse event was triggered
screenY	Returns the vertical coordinate of the mouse pointer, relative to the screen, when the mouse event was triggered
clientX	Returns the horizontal coordinate of the mouse pointer, relative to the current window, when the mouse event was triggered
clientY	Returns the vertical coordinate of the mouse pointer, relative to the current window, when the mouse event was triggered
pageX	Returns the horizontal coordinate of the mouse pointer, relative to the document, when the mouse event was triggered
pageY	Returns the vertical coordinate of the mouse pointer, relative to the document, when the mouse event was triggered
offsetX	the horizontal coordinate of the mouse pointer relatively to the target element
offsetY	the vertical coordinate of the mouse pointer relatively to the target element

MouseEvent object

- **MouseEvent object properties**

Property	Description
altKey	True if the alt key was also pressed
ctrlKey	True if the ctrl key was also pressed
shiftKey	True if the shift key was also pressed
detail	Returns a number that indicates how many times the mouse was clicked
button	Any mouse buttons that are pressed . Possible values: 0: Left mouse button 1: Middle button (scroll wheel) 2: Right mouse button

keyboardEvent object

- A keyboard event is fired when the user press or release a key on the keyboard.

Property	Description
altKey	True if the alt key was also pressed
ctrlKey	True if the ctrl key was also pressed
shiftKey	True if the shift key was also pressed
code	Returns String with the code value of the key represented by the event.
key	Returns String representing the key value of the key represented by the event.

Event object (Cont.)

- **Event Object Methods**

Method	Description
preventDefault()	Cancels the event if it is cancelable, meaning that the default action that belongs to the event will not occur. For example, this can be useful when: • Clicking on a "Submit" button, prevent it from submitting a form • Clicking on a link, prevent the link from following the URL Note: Not all events are cancelable. Use the cancelable property to find out if an event is cancelable.
stopPropagation()	Prevents further propagation of the current event.

Event - preventDefault

- The preventDefault() method cancels the event if it is cancelable, meaning that the default action that belongs to the event will not occur.
- **Event handlers return value:**

```
<a href=“#” onclick=“myFunc();return false”> click me </a>
```

This will make the browser ignore the action of href

- **Another way that can also make the browser ignore the action of href is:**

```
<a href="javascript: void(0)" onclick="alert('hi')" >click me</a>
```

- **Or use preventDefault() method**

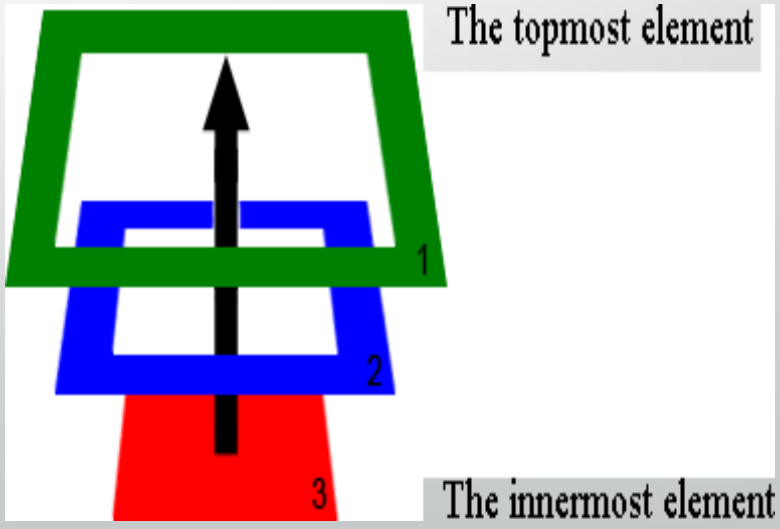
```
<a href=“#” onclick=“myFunc();event.preventDefault()”> click me </a>
```

Event Propagation (Bubbling)

- DOM elements can be nested inside each other. And somehow, the handler of the parent works even if you click on it's child.
- The reason is event bubbling.
- After an event triggers on the deepest possible element, it then triggers on parents in nesting order.

```
<script>
        function tryme()
        {
            alert("document")
        }
        document.onclick=tryme;
</script>
<body>
<input type="text" onclick="alert('input')">
```

if we made a click in the textbox



Event Propagation (Cont.)

- **Cancel Event Bubbling:**

```
<input type="text" onclick="handlerFun(Event)"/>
```

```
function handlerFun(e)
{
        //For most browsers (W3C-compliant browsers)
        e. stopPropagation();
        //For IE <9
         e.cancelBubble = true

}
```

Event handling

1. Event handlers as tag attribute:

```
<input type="button" value=“click me” name="b1" onclick=“alert(‘you
        have made a click’)”>
OR
        <script>
        function showmsg()
        {
            alert(“you have made a click”)
        }
        </script>

<input type="button" value=“click me” onclick=“showmsg()” />
```

Event handling (Cont.)

2. Event handlers as object property:

```
<body>
        <form>
            <input type=“button” id='b1' value="Click ME" />
        </form>
</body>
<script>
        function showAlert ()
        {
            alert(“you have clicked me”)
        }
        document.getElementById(“b1”).onclick=showalert;
</script>
```

Note: there are no parentheses

Event handling (Cont.)

3. addEventListener() method:

- The addEventListener() method attaches an event handler to the specified element.
- The addEventListener() method attaches an event handler to an element without overwriting existing event handlers.
- The addEventListener() method makes it easier to control how the event reacts to bubbling.
- When using the addEventListener() method, the JavaScript is separated from the HTML markup, for better readability and allows you to add event listeners even when you do not control the HTML markup.

Event handling (Cont.)

3. addEventListener() method (cont.):

```
element.addEventListener(event, function);
```

```
document.getElementById(“b1”).addEventListener("click", myFunction);

function myFunction() {
        alert (“Button Clicked");
}
```

- The first parameter is the type of the event (like "click" or "mousedown").
- The second parameter is the function we want to call when the event occurs.

Event handling (Cont.)

3. addEventListener() method (cont.):

- You can easily remove an event listener by using the removeEventListener() method.

```
element.removeEventListener("mousemove", myFunction;(
```

- Note: The addEventListener() and removeEventListener() methods are not supported in IE 8 and earlier versions and Opera 6.0 and earlier versions. However, for these specific browser versions, you can use the attachEvent() method to attach an event handlers to the element, and the detachEvent() method to remove it.

```
element.attachEvent(event, function;(
element.detachEvent(event, function;(
```

JavaScript

THANK YOU